

Amanda Monteiro do Amaral
Orientação: Amanda Lemette
Coorientação: Artur Serpa

Simulação de Monte Carlo para **terpolímeros**: Otimização do tempo computacional



PONTIFÍCIA UNIVERSIDADE CATÓLICA
DO RIO DE JANEIRO





Objetivo

*Diminuir o tempo de simulação
em estado estacionário do
terpolímero em Python*

1

Introdução

Definição de polímeros, suas distribuições, estado estacionário e modelos para simulação.



Introdução

Polímeros

São macromoléculas de cadeias longas com grande peso molecular.

Podem ser sintetizados ou naturais.

Distribuições

Distribuição por peso molecular (MWD) ou Distribuição por comprimento de cadeia (CLD) são as mais prováveis de Flory Schulz.

Estado

Estacionário

Sem atualização de balanço ou mudança de cálculo entre uma cadeia e outra.

Métodos

Determinísticos x Estocásticos



Polímeros

- Homopolímero ou copolímero.
- Polímeros comerciais x naturais.
- Exemplos



Distribuições

- Distribuição por peso molecular (MWD).
- Distribuição por comprimento de cadeia (CLD).
- Média do peso molecular (M_n).
- Média ponderada do peso molecular (M_w).



Estado Estacionário

- Condições iguais do início ao fim da simulação.
- Mesma probabilidade de ocorrência de monômero.
- Ao contrário seria o estado dinâmico.



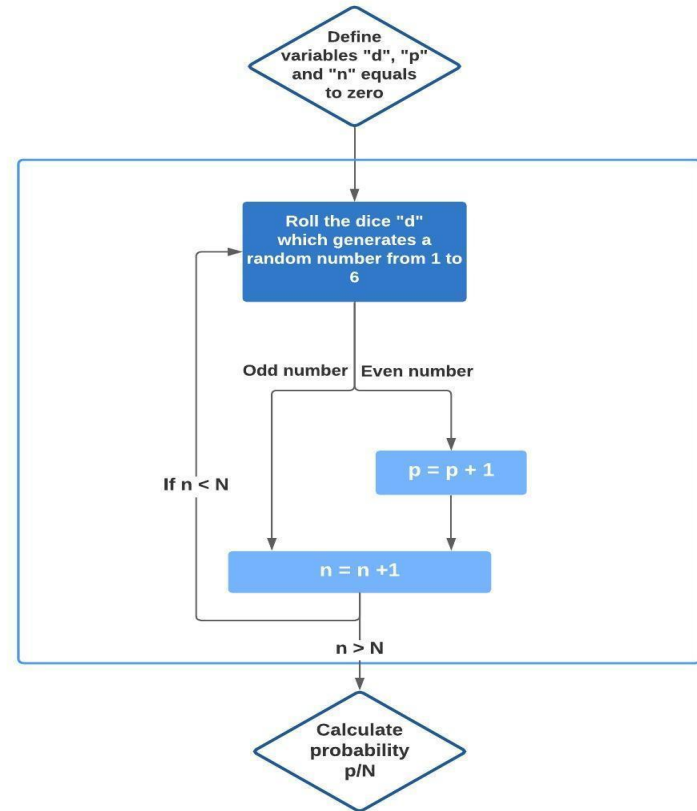
Métodos

- Determinísticos.
- Estocásticos.



Método de Monte Carlo

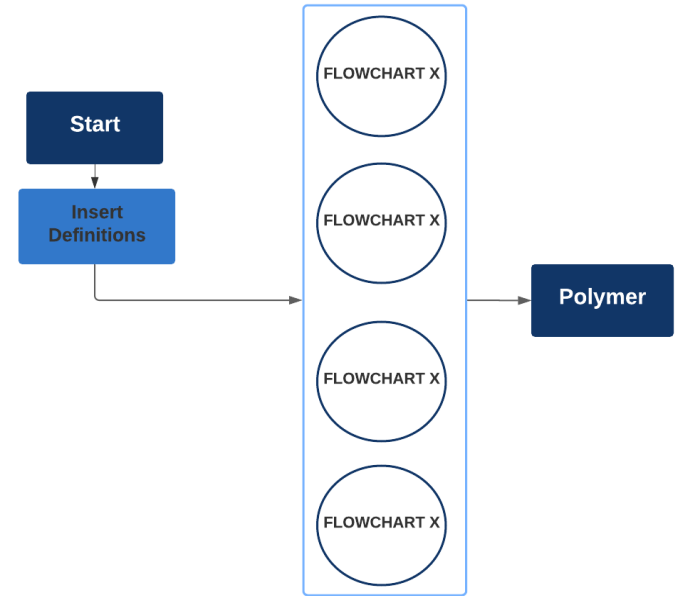
- Baseado em lógica.
- Muito utilizado em copolimerização.





Paralelização como Otimização

- Método de aceleração de código.
- Processamento em paralelo ao invés de em série.





Porque Python?

- Aprendizado simples.
- Linguagem aberta.

2

Bibliografia

Estudos que se utilizaram de Monte Carlo e estado estacionário para simulação de polímeros.



Paralelização em GPU

- Paralelização em processador gráfico.
- Com aplicação de CUDA, plataforma de paralelização computacional.
- Não menciona necessidade de ordenação.

Weng et al. 2015



Método geral para acelerar simulações em MC

- O gráfico de saída também é MWD.
- Testa o algoritmo para diferentes tamanhos para volume de controle.
- Utiliza Numba para otimizar o código.

3

Metodologia

Polimerização, Ordenação e Otimização.

Todas as simulações ocorreram no laptop pessoal com as seguintes configurações: Intel Core i7-10510U CPU @ 1.80Ghz (8 CPUs), ~2.3Ghz 16G RAM. Códigos escritos em Python 3.



Metodologia

Polimerização

Processo de simulação de polímero, feito em Python, estado estacionário e se utilizando do método de Monte Carlo.

Ordenação

Foram testados alguns métodos de ordenação (Bubble, Selection, Insertion e Timsort), buscando encontrar o melhor deles.

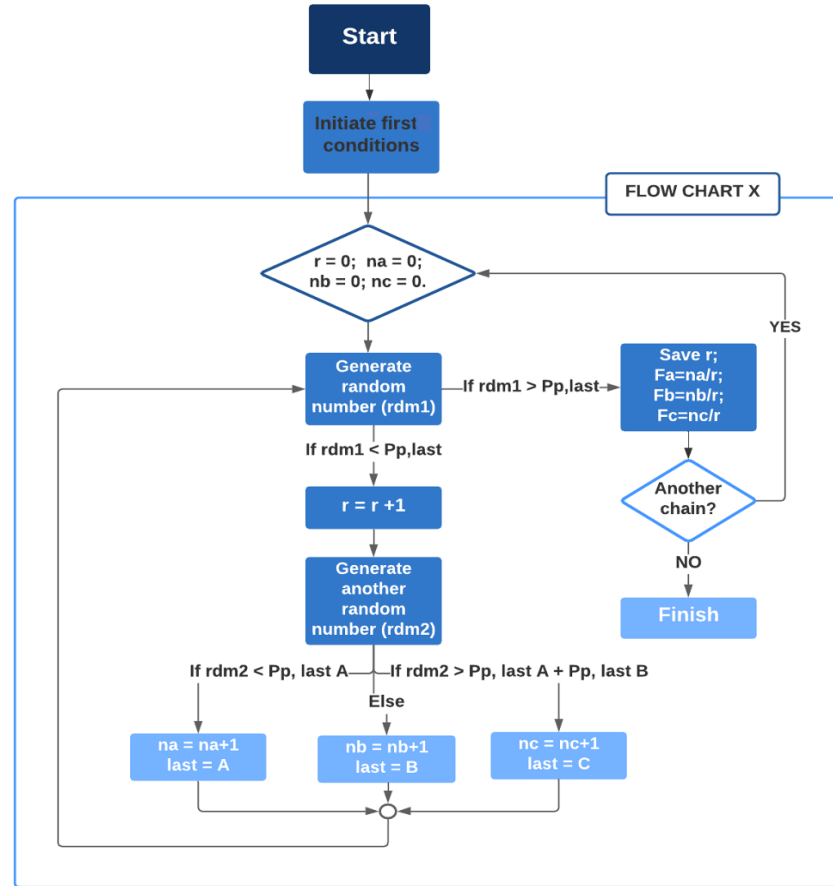
Otimização

Acelera o código, diminuindo o gasto computacional do programa como um todo.



Polimerização

- Fluxo da polimerização.
- A, B e C são os monômeros.
- P_p é a probabilidade de propagação.





Ordenação

- Vetor D contém o tamanho do polímero.
- Fa, Fb e Fc são as frações molares para A, B e C respectivamente.

D	Fa	Fb	Fc
7	0.5	0.33	0.17
1	0.1	0.3	0.6
...	...	...	...
2	0.3	0.7	0
3	0.2	0.5	0.3



D	Fa	Fb	Fc
1	0.1	0.3	0.6
2	0.3	0.7	0
3	0.2	0.5	0.3
...	...	...	...
7	0.5	0.33	0.17



Otimização

- Utilizou-se a biblioteca Joblib.
- Numba, um compilador também foi testado.

4

Algoritmos de Ordenação

Bubble, Insertion, Selection e Timsort.

Todos os algoritmos foram testados 10 vezes se utilizando de mesmo vetor para todos os tipos de ordenação para que a comparação fosse feita de forma justa e com a mesma semente para o número aleatório gerado.



Algoritmos de Ordenação

Bubble

Um dos algoritmos de ordenação mais simples utilizados. Foi o primeiro teste feito.

Insertion

O algoritmo insere no local correto o número adequado e segue ordenando o vetor dessa maneira.

Selection

Essa ordenação ocorre no sentido de sempre procurar o menor número. Ele salva o primeiro elemento e procura algum menor que aquele.

Timsort

Divide o vetor em vários subvetores que são ordenados com o algoritmo Insertion e então agrupados usando o Merge.



Bubble

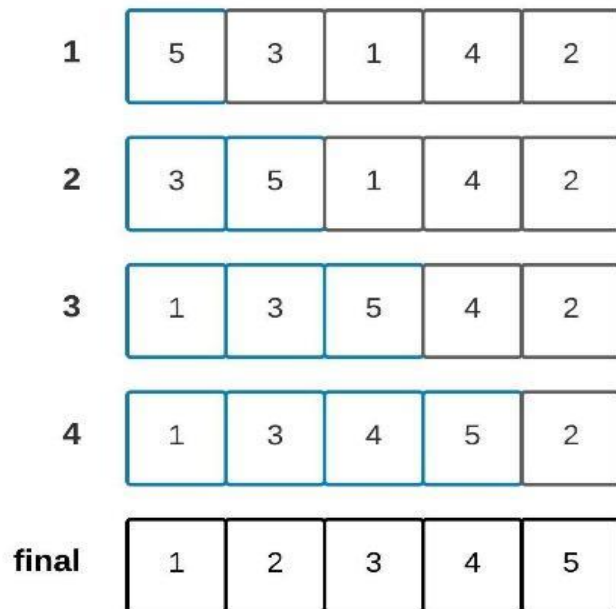
- Segundo a literatura, o algoritmo funciona melhor para vetores pequenos.

1	7	2	5	4	10
2	2	7	5	4	10
3	2	5	7	4	10
4	2	5	4	7	10
5	2	5	4	7	10

6	2	5	4	7	10
7	2	4	5	7	10
8	2	4	5	7	10
final	2	4	5	7	10

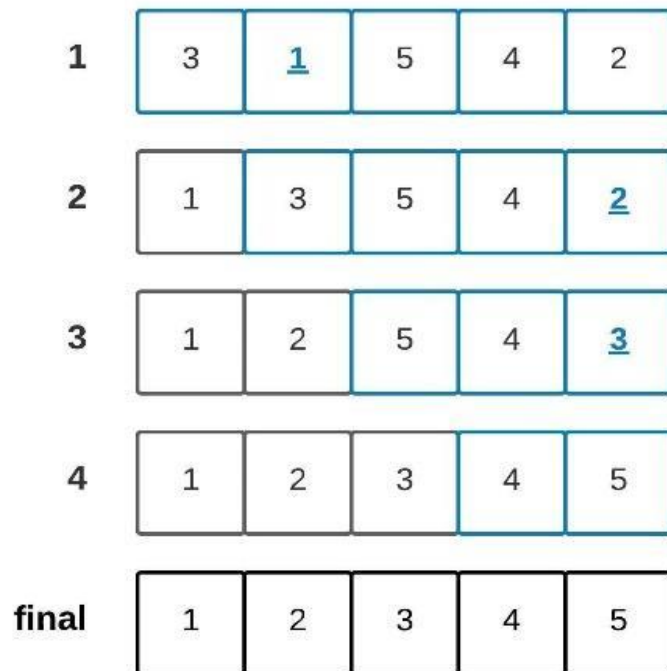


Insertion



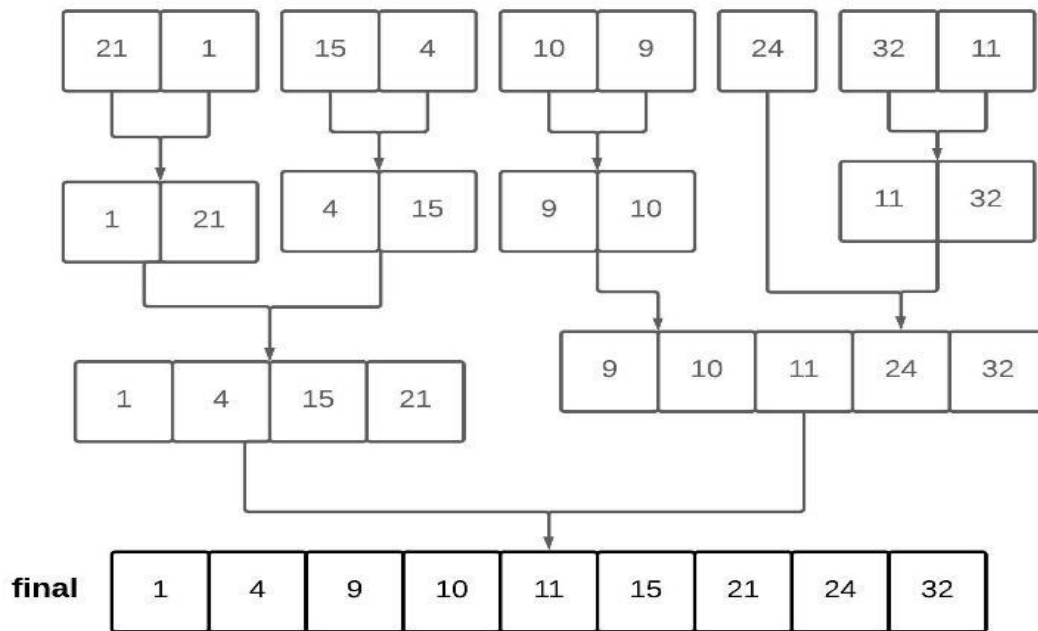


Selection





Timsort



5

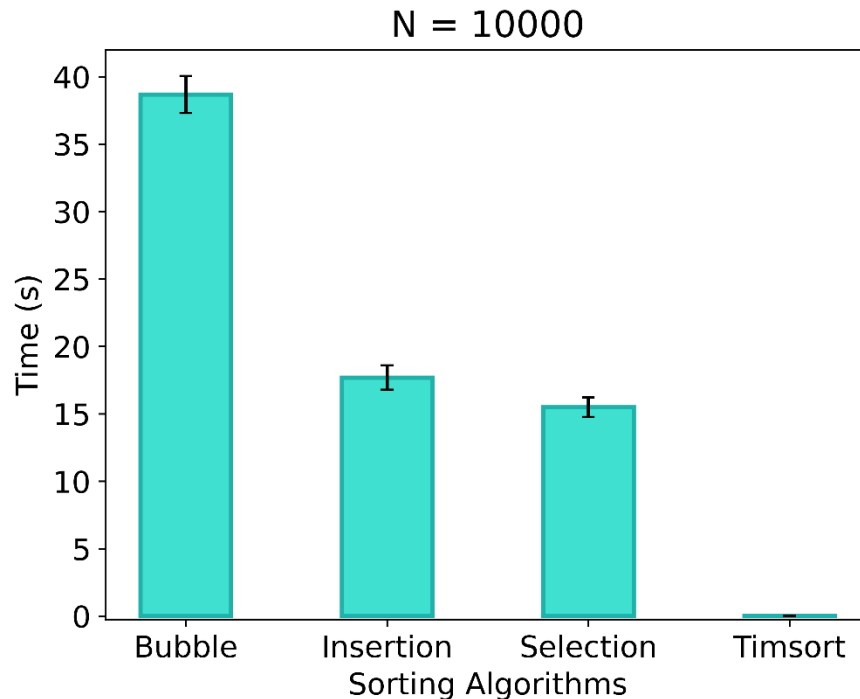
Resultados e Discussões

Máximos tamanhos de cadeia (N) iguais a 500 e 10000.



Comparando ordenações com N = 10000

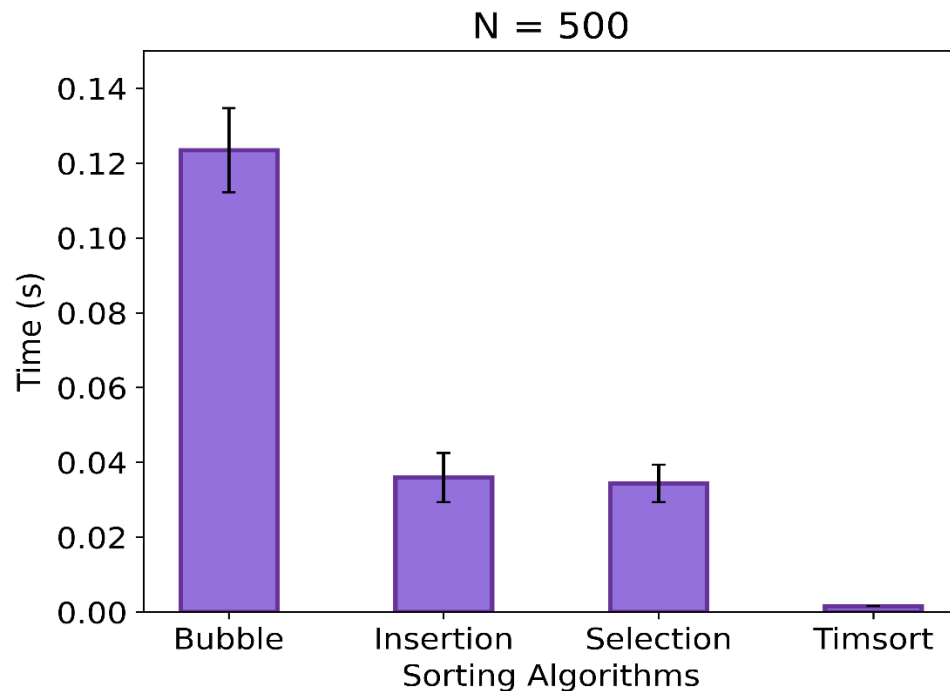
- ⦿ Bubble se destaca como pior disparado.
- ⦿ Timsort já se mostra o melhor algoritmo de ordenação.





Comparando ordenações com N = 500

- Selection e Insertion quase não apresentam diferença de efetividade na ordenação.

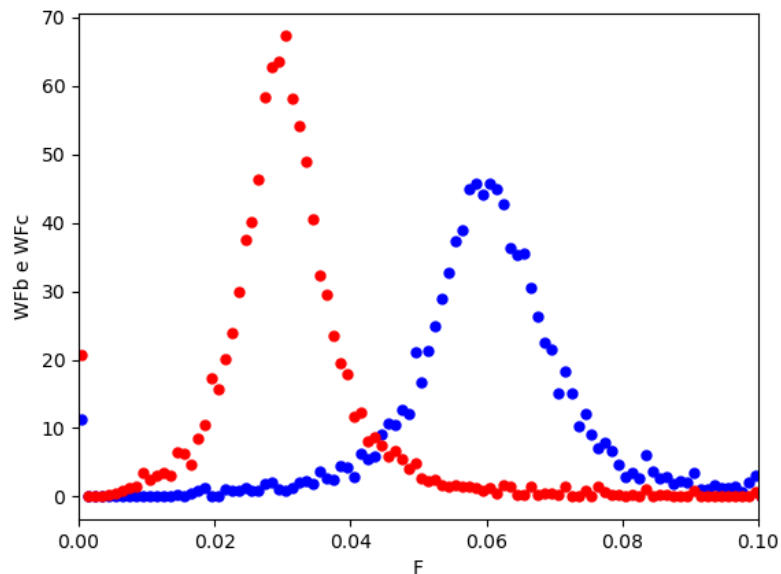


Tomada de decisão de qual o número limitante de cadeias (N)
ideal para a simulação.

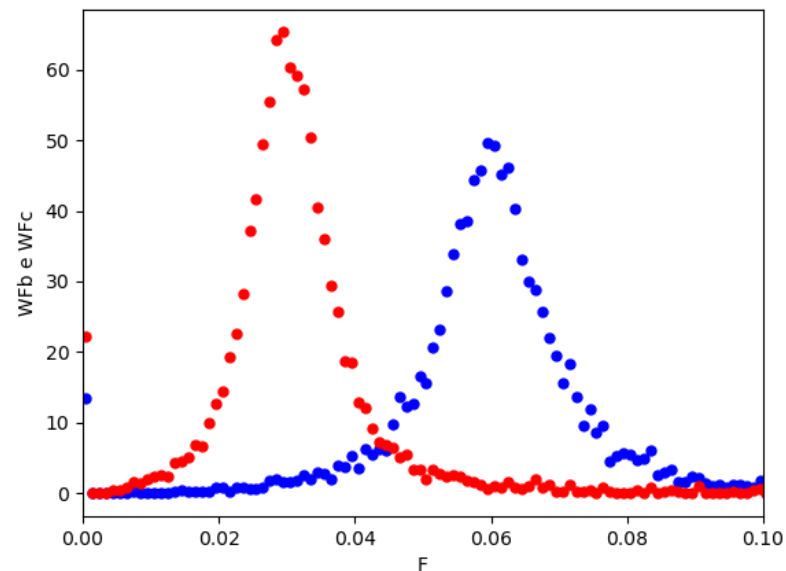


MWD

N = 10000



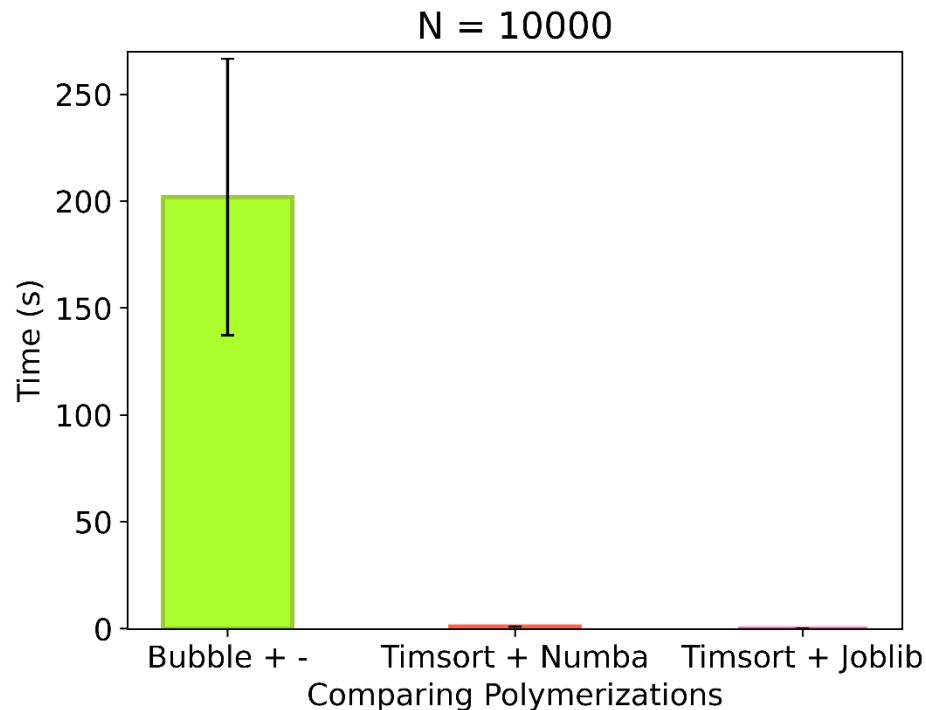
N = 20000

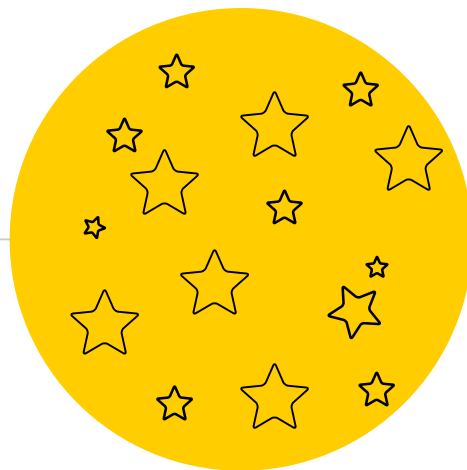




Simulação + Ordenação com N = 10000

- Timsort + Joblib se mostrando a melhor combinação.
- Tempo decai de 200s para quase 0s.





Conclusão



Conclusão

Pior Cenário

Ficou comprovado que o pior cenário testado foi a simulação sem otimização e com a ordenação feita utilizando o algoritmo Bubble, totalizando 202s de tempo computacional.

Melhor Cenário

Timsort junto a otimização da simulação feita pelo Joblib apresentou o melhor resultado, reduzindo o tempo computacional gasto em **99.94%**.

Segunda melhor Otimização - Numba

Aliado ao melhor algoritmo de ordenação – Timsort – se mostrou bem próxima do melhor cenário (Joblib + Timsort).

Categorização dos algoritmos de Ordenação

O estudo entrou em questões de complexidade, explicando as diferenças entre os algoritmos de ordenação estudados.



Obrigada!

Alguma pergunta ?